BSc (Hons) in Computer Science & Artificial Intelligence

Course Work: Critical Review and Technical Baseline of a Machine Learning Research Paper

Reviewed Paper:

"FedTGP: Trainable Global Prototypes with Adaptive-Margin-Enhanced Contrastive Learning for Data and Model Heterogeneity in Federated Learning"

Zhang, J., Liu, Y., Hua, Y., & Cao, J. (2024).
Publication: Proceedings of the AAAI Conference on Artificial Intelligence, 38(15), 16768-16776.
DOI: https://doi.org/10.1609/aaai.v38i15.29617

- Student Name : W.A.Thisumi Nimsani
- Student ID : KIC-BSCCSAI-251P-030
- Module : Machine Learning Related Application
- Supervisor / Module Leader : Mr.Ashan Arthnayake
- Date of Submission: Oct 23, 2026

Table of Contents

Phase 3: The Technical Review Report **3**
A. Technical Summary **3**
A.1 Problem and main contribution 3
A.2 Methodology and learning pipeline 3
A.3 Evaluation and practical findings 4
B. Methodological Critique **5**
B.1 How well is the problem solved? 5
B.2 Do the evaluation criteria meet the needs? 6
B.3 Checkpoint selection and the strength of the statistical evidence 6
B.4 Assumptions made for the prototype representation 7
B.5 Computational costs and practical results 8
B.6 Privacy and overall judgement 8
C. Proposed Alternatives **9**
C.1 Multiple prototypes per class 9
C.2 Robust prototype Aggregation (Geometric median) 10
C.3 How the alternatives should be evaluated 11
D. Stress-Testing **12**
D1. Research question and hypothesis 12
D.2 Experimental design 13
D.3 Evaluation procedure 16
D.4 Classification accuracy and consistency 17
D.5 Computational cost and worst-client performance 19
D.6 Interpretation and limitations 22
Conclusion **24**
Implementation and Evidence **24**
References **25**

Phase 3: The Technical Review Report

This review evaluates the evidence supporting FedTGP, proposes two alternatives, and reports an architecture stress test. The completed experiments favour FedTGP under the tested conditions, while showing additional computational cost and limits to generalization.

A. Technical Summary

A.1 Problem and main contribution

FedTGP tackles image classification in a supervised setting, with heterogeneous federated learning, where clients may have different data distributions and neural network architectures. These clients are unable to directly merge their full model parameters if their architectures are incompatible. Instead, they trade class prototypes—small vectors which summarise the average extracted features of locally available examples belonging to each class [1].

The main contribution of the paper is related to how these prototypes gain their usefulness as global knowledge. Averaging prototypes across different clients can decrease the separation between classes and produce less informative targets for local learning. Therefore, in FedTGP, the learning of global prototypes is done by way of an explicit optimisation objective, not just averaging. The goal is to keep global prototypes close to local prototypes of the corresponding class while improving their separation from prototypes of other classes [1]. The problem formulation and data pipeline were examined in the phase 1 resubmission [2]. This section briefly summarizes the method before evaluating its strengths and limitations.

A.2 Methodology and learning pipeline

There is a feature extractor and a classification head for each client. The feature extractors produce feature vectors with the same dimension, set to 512 by default, even though the architectures of the clients vary. Local training is a combination of classification loss and a prototype-alignment term. The classification loss aligns learning to the appropriate image label,

and the alignment term provides guidance from the shared global prototypes. Clients next compute prototypes for the classes present in their local data and upload them to the server [1].

On the server, a trainable vector for each class goes through two fully connected layers, the activation function in between being ReLU. The optimised global prototypes are obtained by adaptive-margin-enhanced contrastive learning (ACL). This objective encourages each uploaded prototype to be closer to its corresponding global class prototype than to the global prototypes of other classes. Adding a margin to the correct-class distance makes this matching task more demanding, encouraging a stronger distance advantage. The margin adapts during training and is capped to prevent excessively large values [1].

The global prototypes guide subsequent local training, while the updated local prototypes provide information for further server-side learning. This creates a feedback loop between clients and the server. Their function does not stop at training, but is also used to classify during inference, where an image is assigned to the class whose global prototype is closest to the image's extracted features. Hence, the quality of the prototype will affect the guidance given to the client as well as the final classification decision [1].

A.3 Evaluation and practical findings

Under two non-IID settings, the paper compares FedTGP with six baseline methods on the CIFAR-10, CIFAR-100, Flowers102 and Tiny-ImageNet datasets. By default, it uses 20 clients, 1,000 communication iterations and three trials. For each trial, the authors choose the best average client-test result and then report the mean and standard deviation of these three selected results [1].

The Phase 2 experiment compared FedProto and FedTGP with the same saved CIFAR-10 client partitions, employing 20 clients, eight different architectures and 100 communication updates for each method. FedTGP scored a best recorded sample-weighted test accuracy of 83.46%, while FedProto scored 71.79%. However, its reported runtime was about 37% higher, and its accuracy fluctuated across recorded evaluations [3].

These findings highlight the accuracy–computation trade-off but do not constitute an exact reproduction of the paper. The implementation used one trial and an alignment weight of $\lambda = 10$ [3], compared with three trials and $\lambda = 0.1$ reported in the paper [1]. The following methodological critique examines what conclusions can reasonably be drawn from the published evidence and these practical findings.

B. Methodological Critique

B.1 How well is the problem solved?

FedTGP tackles an important drawback of heterogeneous federated learning, which is that averaging prototypes from heterogeneous clients may produce less discriminative global representations. It is possible to average the class prototypes if several clients have different neural-network architectures, but averaging the representations does not necessarily yield useful global learning targets. The arrangement of feature spaces can vary between different clients, and averaging can reduce the distance between prototypes of different classes. In response, FedTGP learns global prototypes by using an adaptive-margin contrastive objective [1].

This is also a reasonable methodological choice because the server optimizes the representations that the clients use for guidance. But, greater distance between global prototypes is not a guarantee that all of a client's image representations are also well separated. Local training is also important for classification because the representations it produces must correspond to the embedded prototypes; and the quality of the local feature extractor determines the quality of the embedding. Thus FedTGP is intended to promote improved class separation, but does not ensure it.

The paper offers evidence based on comparisons over four datasets of images and two non-IID settings and a few configurations of heterogeneous models. It also contains ablation experiments in which the effects of the proposed components [1] are studied. This is a more robust comparison than one that is based on only one set of data. However, the findings only are applicable to the image-classification configurations that were evaluated and are not necessarily applicable to other federated-learning applications, such as text or audio.

B.2 Do the evaluation criteria meet the needs?

The primary measure of classification accuracy is an appropriate measure in this case because the task involves predicting one correct catgory for every image. It also makes a simple comparison with the baseline methods. But when the clients have unequal class distribution and unequal amounts of data, an overall accuracy value will not suffice.

For instance, a sample-weighted accuracy would allow larger test sets to have more influence on the final result. A high value may therefore go hand in hand with poor performance on smaller clients. An unweighted mean gives each client equal influence, although accuracy estimates from very small client test sets can be unstable. These measures represent different questions and are more informative when they are reported together.

Macro-F1, class-level recall and the distribution of client accuracies would provide further evidence about who benefits from the method. Reporting the lowest-performing clients would help establish whether higher overall accuracy is accompanied by more consistent performance across the federation.

The Phase 2 experiment is an example of the significance of the distinction. FedTGP improved its best recorded sample-weighted test accuracy compared to FedProto, but this result should be taken with the client-level results and learning curves [3]. A higher overall score indicates an improvement under that evaluation rule , but does not demonstrate that every client benefits.

B.3 Checkpoint selection and the strength of the statistical evidence

For each trial, the paper shows the best average accuracy of the client tests achieved during training, along with the mean and standard deviation of the three trials. Repeated trials are useful because they provide some indication of variability. The very selection of the best one with the test set, however, makes the test set part of the selection process [1].

This is not to say that test images were used directly to compute training gradients. The worry is that frequent inspection of test performance and reporting of the best performance may give an inflated sense of generalisation. An improved evaluation would be to use a different validation

set to choose the communication round and then use the test set to evaluate the chosen model. Or, one could report performance at one of the specified final rounds, which would not require the selection of the max test score.

The same applies to the Phase 2 comparison, which provides best recorded test accuracies from a single run [3]. Therefore, it is useful as an implementation study but it is not sufficient to prove the observed gain is repeatable with random seeds. Furthermore, changes from one round of communication to the next are not, in themselves, indicators of overfitting. They can also be caused by prototype alignment changes, local optimisation, or the effects of client heterogeneity.

To show that the overfitting had occurred, further evidence would be needed, like the fact that the training performance was steadily diverging from the independent validation performance.

B.4 Assumptions made for the prototype representation

FedTGP provides a common feature dimension which enables prototype exchange between different architectures. However, equal dimensions does not mean that the coordinates will be interpreted in the same way by the clients. The local alignment goal helps clients learn representations that align with the global prototypes, yet they must learn to align during training.

Another restriction is that there is one prototype for each class. There can be multiple meaningful groups within the same category, due to viewpoint, background or appearance differences. A single representative may not adequately capture this variation. Although FedTGP improves how that representative is learned, it does not explicitly model multiple distinct groups within each class [1].

Prototype reliability also depends on local class support. A prototype estimated from very few examples can be unstable, while a client with many examples may still produce a poor representation if its feature extractor is weak. Therefore, neither sample count nor distance alone provides a complete measure of prototype quality. This is especially relevant under strongly non-IID partitions, where some classes are represented on only a few clients.

The adaptive margin acts as a soft training constraint. It encourages the correct global prototype to have a distance advantage over competing classes, but does not mathematically guarantee that all class pairs remain separated. The normalised margin illustrations in the paper aid in the communication of the authors' argument, but should not be viewed as a direct comparison of the absolute distance between methods [1].

B.5 Computational costs and practical results

Prototype exchange can reduce communication compared with transmitting entire neural-network parameter sets. However, there is a difference between the efficiency of communications and the efficiency of computation. FedTGP introduces server-side optimization of global prototypes, whereas FedProto aggregates uploaded local prototypes.

FedTGP obtained the highest accuracy of 83.46% in the Phase 2 experiment and FedProto achieved 71.79%, which is 11.67 percentage points lower. It was recorded that the duration of its run was about 37% longer [3]. These results indicate an accuracy–computation tradeoff in the applied settings. The paper also has a discussion on computational efficiency comparisons of time-to-target [1]. A method may require more time per round of communication and still achieve a desired level of accuracy in a shorter time. Thus, the time span of the longer run does not necessarily contradict the efficiency results from the paper. Accuracy under a fixed training budget and time to a specified accuracy should be reported separately.

The practical experiment is not an exact reproduction of the published evaluation. It used a smaller training budget, one trial and an alignment weight of $\lambda = 10$, whereas the paper reports three trials and a default value of $\lambda = 0.1$ [1] [3]. Such differences make it difficult to attribute any discrepancies between the actual and published results to implementation quality or to the performance of the algorithms alone.

B.6 Privacy and overall judgement

Prototype sharing is not a formal privacy assurance, but keeping raw images on clients minimizes the need to transmit sensitive information. The information remaining in prototypes is

based on local examples. Establishing resistance to information leakage would require a clear threat model and suitable privacy or attack evaluations.

Overall, the paper is providing credible experimental evidence in favor of learning global prototypes rather than simply averaging them. The comparisons and ablations demonstrate its usefulness in the tested environments [1]. The primary qualifications are related to selecting test-based results and not much evidence on different client performances, assumptions about the representations and the difference between communication savings and computational cost. Results from this Phase 2 further justify research into FedTGP, but are too limited to establish superiority across all federated-learning settings [3].

C. Proposed Alternatives

Below are alternatives to FedTGP for various shortcomings. These are suggested adaptations and not ones that have been put into practice or tested in this coursework. Each would need a controlled comparison before any improvement could be claimed.

C.1 Multiple prototypes per class

An alternative is to use a small number of prototypes for each class rather than a single representative for the entire class. Images in the same category may vary widely in appearance. If they are encouraged to go to the same prototype, it is possible that important variation within the class is missed.

The motivation of the proposal comes from Infinite Mixture Prototypes for Few-Shot Learning: several clusters are used to represent categories [4]. This work gives the motivation to explore more flexible class representations, but the results of few-shot learning is not enough to guarantee the same outcomes in heterogeneous federated learning.

In the proposed adaptation, the feature vectors from each client will be grouped together in each class. The extra cost could be capped by having two or three prototypes for each class. If there

are not enough examples in a class, one prototype will be maintained. Validation data would be used to choose the number of prototypes and minimum number of examples per cluster.

The server would store a number of global prototypes per class. The prototypes would be uploaded and each one would be matched to its closest global prototype of its kind, instead of clustering them by their index of prototype. This is because, 'cluster 1' on one client does not necessarily represent the same visual pattern as in another.

Local alignment would attract an image representation to its most similar global representation of the same class. During inference, classification would use:

$$\hat{y} = \ argmin_c min_{r \in \{1,\dots,m_c\}} \left|\left| f_i(x) - P_{c,r} \right|\right|_2^2 \quad (1)$$

Here, m_c is the number of global prototypes for class c .

The goal is to have a better class representation of several meaningful feature groups. But other prototypes can capture noise or be funneled to the same destinations. This way, the need to coordinate representations between architectures is not eliminated. This would require modifying FedTgP's contrastive objective and local alignment loss to support multiple same-class prototypes.

The prototype-vector payload would grow from K to $m_c K$ per class. Also there would be extra computation for clustering and matching. The alternative is therefore designed to enhance the quality of the representation, and not necessarily the efficiency.

C.2 Robust prototype Aggregation (Geometric median)

The second alternative would be to use a robust class-wise aggregation instead of FedTGP's trainable global-prototype generator. However, in non-IID learning, some prototypes can be unreliable due to being estmated by using small class samples or due to poorly trained local models. An unusually distant vector may give a strong influence to the arithmetic mean.

Pillutla, Kakade and Harchaoui explore geometric-median aggregation for federated learning [5].

Their work relates to model updates; class prototypes would be another adaptation.

The server might be able to obtain a representative from class c by estimating:

$$P_c^* = argmin_p \sum_{i \in I_c} w_{i,c} \left|\left| p - P_{i,c} \right|\right|_2 \quad (2)$$

The set I_c is the set of clients that contribute and the weights are non-negative with sum 1.

The geometric median is different from the arithmetic mean as it minimises unsquared Euclidean distances. This reduces its sensitivity to very far away contributions. It could allow for more stable global learning targets when a minority of uploaded prototypes are less reliable.

However, an unusual prototype is not necessarily incorrect. It may represent a legitimate minority group or a client with differing visual conditions. That information might be suppressed by robust aggregation. Also, there is no explicit separation across types or classes and no semantic differences across client feature spaces are resolved.

Replacing FedTGP's generator with this procedure could remove the need for server-side neural-network training. However, geometric-median estimation also requires computation, so any efficiency benefit would need to be measured.

C.3 How the alternatives should be evaluated

For each alternative, it must be compared to the original single-prototype FedTGP separately with the same client partitions, architectures, initialisation seeds and training budget. The validation data is used to inform the configuration decisions, and the test set is reserved for final performance evaluation.

Alternative	Expected benefit	Main Limitation
Multiple prototypes (C.1)	Represent several feature groups within a class	More communication and clustering; possible prototype collapse
Geometric median (C.2)	Reduce sensitivity to unreliable contributions	May suppress legitimate minority patterns; no explicit class separation

Table C.1: Expected benefits and main limitations of the proposed adaptations.

Sample-weighted accuracy is recommended to be evaluated as well as macro F1 score, individual client performance, prototype bytes transmitted and runtime. Multiple random seeds would help to differentiate between a consistent improvement or a lucky individual run. It is also possible to compare them in terms of fixed run-time to see if there is any gain in accuracy worth the added computation.

The alternatives would be considered only if they would still have worthwhile advantages under those considerations. Their expected benefits remain research hypotheses and have not been measured in this present course work.

D. Stress-Testing

D1. Research question and hypothesis

The following research question was explored in this experiment: What happens to the classification accuracy and runtime of FedTGP and FedProto when the composition of the client architecture changes?

Heterogeneous Federated learning allows for the use of distinct neural networks due to varying computational capabilities and application needs among clients. These networks can learn to create feature vectors of the same dimension but they may learn different representations of the same image category. This raises a practical question: does the way global prototypes are constructed help clients continue learning effectively when the architecture mixture changes?

The hypothesis was that FedTGP would be more consistent in classification accuracy than FedProto in architecture configurations selected. In FedTGP, the global prototypes are learned from the available prototypes with an adaptive-margin contrastive objective [1] which is different from FedProto, which averages the uploaded prototypes separately for each class. This provides a reason to expect FedTGP to handle differences between local representations more effectively. But the experiment was required to see if the expectation was fulfilled under the specified conditions.

The previous Phase 2 experiment evaluated the methods with a single set of client architectures [3]. This present experiment took that assessment a step further and evaluated three architecture configurations on the same five-client data partition. It explored accuracy achieved and computation required, as reaching a higher degree of accuracy at an appreciably greater cost could have different practical implications.

This hypothesis did not assume that adding more architecture types would always reduce accuracy. A replacement network might be better suited to a particular client's data than the original network. The experiment therefore examined how the two methods responded to the same architecture changes, without assuming that accuracy would necessarily decline.

D.2 Experimental design

The participation ratio was 1.0, with five clients participating in each communication round. Full participation ensured that changes in the participating client group did not introduce another source of variation. Five clients and their saved data were used for each configuration, with the network architectures varied.

S1 did provide a reference configuration with all clients running ResNet34. Client 4 was running MobileNetV2 in S2 and the other clients were running ResNet34. In S3, client 3 used ResNet18 while client 4 retained MobileNetV2, producing a mixture of three architecture types. These were independent training sessions and models were not changed during the course of an on-going training run.

Client	S1: One Architecture Type	S2: Two Architectures Types	S3: Three Architectures Type
0	ResNet34	ResNet34	ResNet34
1	ResNet34	ResNet34	ResNet34
2	ResNet34	ResNet34	ResNet34
3	ResNet34	ResNet34	ResNet18
4	ResNet34	MobileNetV2	MobileNetV2

Table D.1. Client architecture assignments for the stress test.

In both methods and all configurations and training seeds, one saved 5-client CIFAR-10 partition was reused. The partition employed the Dirichlet concentration parameter $\alpha = 0.1$, favoring unequal distribution of classes among the clients. There were also significant variations in sample size between clients in the realised partition. Despite the use of a common network architecture, there was statistical heterogeneity in S1.

Reusing the saved allocations was crucial as it would alter the architecture mix and the data available to the clients, if a new partition was created for each configuration. A fixed partition facilitated the interpretation of differences related to the architecture assignments.

The implementation concatenated the 60,000 images of CIFAR-10 together and then split them into local training and test sets. The results therefore describe the saved federated partition rather than an evaluation using CIFAR-10's original train–test split.

Client	Examples for Training	Examples for Testing
0	453	151
1	10,005	3,335
2	11,681	3,894
3	9,842	3,281
4	13,018	4,340
Total	44,999	15,001

Table D.2. Number of training and test examples given to each client.

The client counts were obtained from the saved partition outputs in the stress-test notebooks.

The evaluation was performed based on training seeds 0, 1 and 2 with 100 communication rounds per run. The whole experiment was thus comprised of:

$$3\,configurations \times 3\,seeds \times 2\,methods = 18\,runs$$

This budget of 100 rounds was a reasonable number to allow for comparison of the methods in the limited computational time available. This was a fixed experimental budget and did not establish that every model had fully converged. The accuracy and runtime were compared after the same number of communication rounds with the same budget.

Corresponding FedProto and FedTGP runs began with matching initial client-model weights. Within each method, the saved partition and training hyperparameters remained fixed across S1–S3. These controls reduced variation caused by different starting conditions and data allocations. However, replacing an architecture also changed model capacity and computational requirements, so the experiment did not isolate architectural diversity from other model properties.

D.3 Evaluation procedure

Both methods were evaluated after each completed communication round, with the main comparison based on the results after round 100. The experiment did not select the highest test accuracy observed during training. Selecting the highest test score can produce an optimistic estimate of performance. Reporting the result after a predetermined number of rounds avoids choosing a checkpoint solely because it performed best on the test set.

The main classification metric was sample-weighted test accuracy. The ratio of correct predictions made by all the clients to the number of test examples was computed. This is the percentage of data images classified correctly in the federation. Clients with larger test sets therefore contribute more strongly to the result. For instance, client 4's test examples were 4,340, and client 0's were only 151. Thus, their effects on the overall accuracy were not equal.

In addition to this overall measure, the evaluation also took into account the worst-client accuracy. This was the lowest accuracy for all five clients for each run. The minimum value that was reported for each of the three seeds was then averaged and used as the reported summary for the three seeds. This helped assess whether stronger overall performance was accompanied by better performance among the weakest clients. The worse-performing client might vary across runs.

Recorded time included training and evaluation. It was a practical indicator of how long it takes to finish the set communication budget, not the entire Colab session including downloads of data and setup of the environment. As both methods were simulated in a sequential way on a GPU, these measurements depict the execution cost of the implementation, not the wall clock behaviour of a real federation of clients running in parallel.

Learning curves recorded how test accuracy changed during training. They helped to differentiate between long-term gains and short-term fluctuations. But variations from round to round were not considered as evidence of overfitting as it may occur due to local optimisation or changes in prototype alignment.

The final accuracy and run-time for each configuration and method were summarized by the mean and the sample standard deviation within the three training seeds. The average was the mean and the standard deviation was the spread of the results from one run to another. These statistics measured training variation on one fixed partition. They did not measure variation across different client allocations, and the standard deviations should not be interpreted as confidence intervals.

D.4 Classification accuracy and consistency

FedTGP achieved higher mean final test accuracy than FedProto in all three architecture configurations. The differences were 10.91 percentage points in S1, 15.42 percentage points in S2 and 12.81 percentage points in S3. The results presented here indicate that this advantage existed within the same as well as different architectures.

Configuration	FedProto (%)	FedTGP (%)	Gain (pp)
S1	68.34 ± 2.42	79.25 ± 0.68	10.91
S2	62.92 ± 1.58	78.34 ± 0.83	15.42
S3	66.21 ± 3.30	79.02 ± 0.44	12.81

Table D.3: Final test accuracy after 100 completed rounds; mean ± sample SD over three seeds.

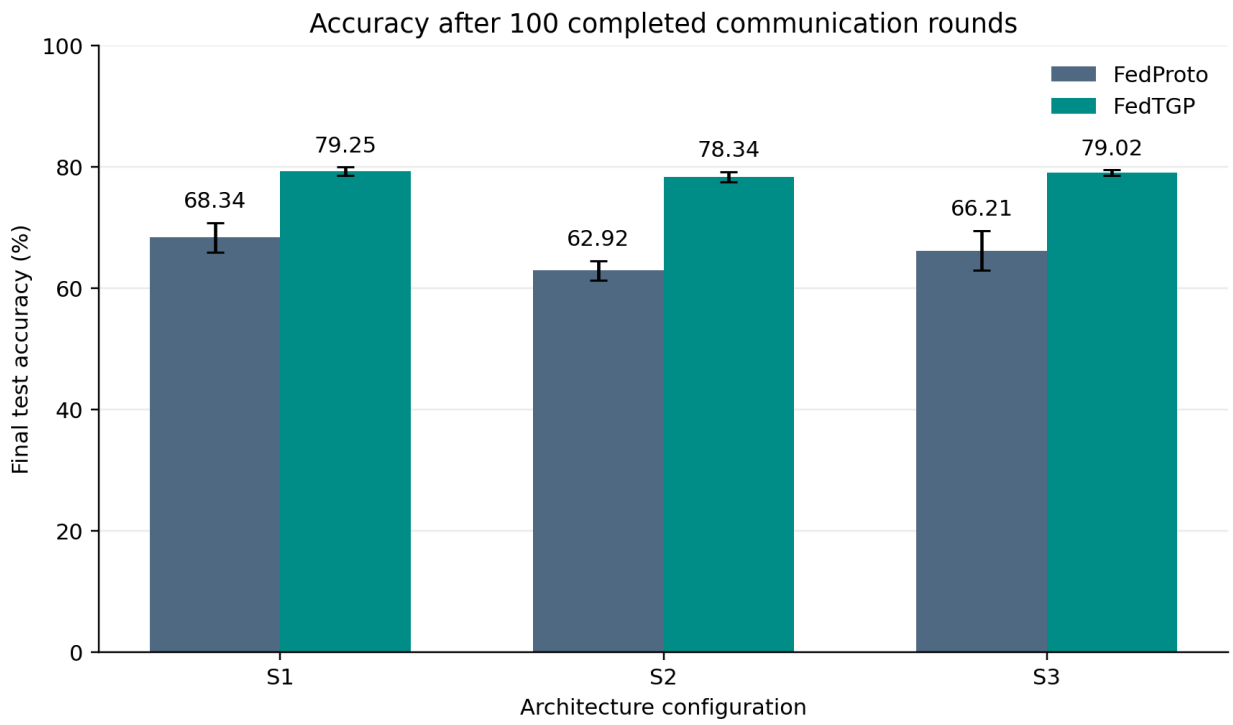


Figure D.1. Final test accuracy after 100 completed communication rounds, with weights applied to the samples. Error bars are sample standard deviations and bars indicate means across three training seeds.

The S1 result is particularly useful for interpreting the comparison. All five clients were trained using ResNet34 but each had a different data distribution locally, meaning they trained their models independently. Using the same architecture therefore did not make their learned feature representations identical. FedTGP's stronger performance in this setting suggests that the usefulness of learned global prototypes is not limited to differences in network architecture. However, this comparison alone cannot establish which part of the learning process produced the improvement.

The largest difference between the methods occurred in S2. The accuracy went down by 5.42 percentage points when replacing client 4's ResNet34 with MobileNetV2 resulting in a mean accuracy of 62.92% for FedProto. FedTGP's accuracy decreased by 0.91 percentage points, from 79.25% to 78.34%. Under this particular assignment, FedTGP was therefore less sensitive to the architecture change.

This replacement affected a substantial part of the evaluation. Client 4 had 4,340 of the 15,001 test examples, accounting for approximately 28.9% of the total. Its predictions consequently had considerable influence on sample-weighted accuracy. Changes to this client could also affect the prototypes shared with other clients. The observed difference should therefore be interpreted in relation to both the architecture replacement and the amount of data assigned to that client.

Performance improved for both methods in S3, when client 3's ResNet34 was replaced with ResNet18. FedProto increased from 62.92% to 66.21%, while FedTGP increased from 78.34% to 79.02%. This finding does not support a simple assumption that adding more architecture types necessarily reduces accuracy. A different architecture may interact more favourably with a client's data or with the shared representations. Further analysis would be required to explain the specific cause of the improvement.

FedTGP also showed less variation across the three training seeds. Its standard deviations ranged from 0.44 to 0.83 percentage points, compared with 1.58 to 3.30 percentage points for FedProto. The largest variation occurred for FedProto in S3. This indicates that the final performance of FedProto was more sensitive to the training seed in that configuration.

Across configurations, FedTGP's mean accuracy varied by only 0.91 percentage points between its highest and lowest results, whereas FedProto's varied by 5.42 percentage points. Together, the smaller differences between configurations and the lower variation across seeds provide evidence of more consistent final performance for FedTGP in this experiment. They do not, however, establish statistical significance or guarantee the same behaviour on other data partitions.

D.5 Computational cost and worst-client performance

The accuracy improvement came with an additional computational cost. FedTGP required approximately 39.2% more recorded time in S1, 41.1% more in S2 and 33.0% more in S3. For example, the mean runtime in S2 increased from 3.19 hours for FedProto to 4.50 hours for FedTGP. The practical question is therefore whether the improvement in classification performance justifies the additional training time.

Configuration	FedProto (h)	FedTGP (h)	Extra time (%)
S1	3.10 ± 0.01	4.31 ± 0.07	39.16
S2	3.19 ± 0.01	4.50 ± 0.05	41.13
S3	2.90 ± 0.03	3.86 ± 0.05	32.97

Table D.4: Recorded training and evaluation runtime; mean ± sample SD in hours.

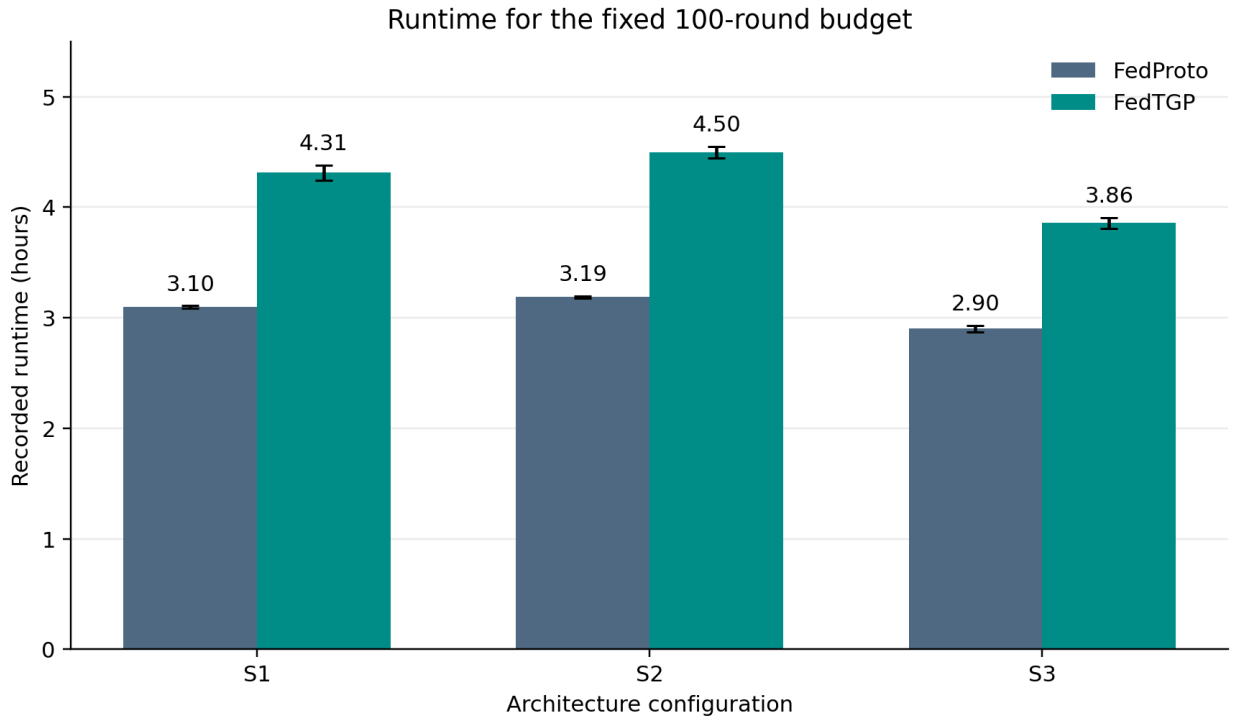


Figure D.2. Timed and recorded training and evaluation for 100-round experiments. The mean runtime is shown by bars, with sample standard deviations for three seeds. These times refer to the sequential GPU simulation.

A likely contributor is FedTGP's server-side optimisation. FedProto aggregates the uploaded prototypes, whereas FedTGP performs additional optimisation to learn its global representatives. However, the recorded runtime covers training and evaluation together. It does not separate local training, prototype collection, server optimisation and evaluation, so the experiment cannot attribute the entire runtime difference to the server.

The timing results also show why model names or parameter counts alone are insufficient for predicting execution speed. S2 took longer than S1 for both methods, even though one ResNet34 was replaced with MobileNetV2. Actual runtime depends on how operations execute on the GPU, the amount of data assigned to each model and the surrounding implementation. A more detailed efficiency study would measure these components separately.

This experiment compared the methods under the same number of communication rounds. It therefore answers how much accuracy and computation each method produced within a fixed training budget. It does not determine which method would reach a chosen accuracy first. That would require a separate time-to-target comparison using a predefined target.

Worst-client accuracy provides another perspective on whether the additional computation was useful. FedTGP's mean worst-client accuracy remained between 73.63% and 74.85% across the configurations. FedProto's corresponding values ranged from 49.32% to 57.38%. The advantage therefore extended beyond the overall sample-weighted result to the weakest client performance within each run.

Configuration	FedProto (%)	FedTGP (%)
S1	57.38	73.63
S2	49.32	73.78
S3	56.42	74.85

Table D.5: Mean of the worst-client accuracy from each run.

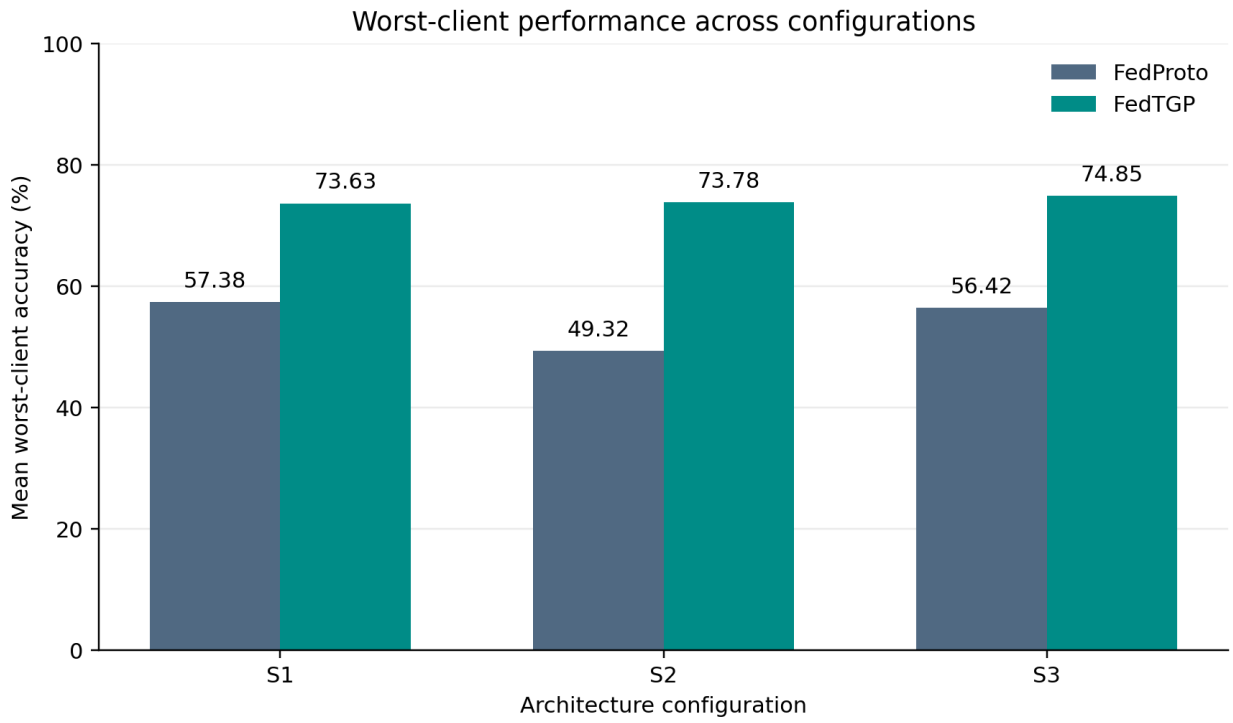


Figure D.3. Mean worst client accuracy over three seeds. The worst-performing client may differ between runs.

This is relevant because the clients had very unequal test-set sizes. Client 0 had only 151 test examples, approximately 1.0% of the total, while client 4 had 4,340. Poor performance on a small client could have only a minor effect on overall sample-weighted accuracy. Examining the worst client makes such weaknesses more visible.

Nevertheless, the mean worst-client metric has limits. The identity of the worst-performing client can change between seeds, and the metric does not show the complete distribution of client accuracies. Higher mean worst-client accuracy therefore supports stronger performance at the lower end of the distribution, but it does not demonstrate that every client improved or that performance became equal across clients.

D.6 Interpretation and limitations

The findings support the hypothesis within the tested architecture assignments. FedTGP achieved higher mean final accuracy, lower variation across training seeds and higher mean worst-client accuracy in all three configurations. The results are consistent with the proposed benefit of

learning global prototypes, although a performance comparison alone cannot prove the precise mechanism responsible for the improvement.

Finding	FedProto	FedTGP	Interpretation
Mean final accuracy (S1)	68.34%	79.25%	+10.91 pp
Mean final accuracy (S2)	62.92%	78.34%	+15.42 pp
Mean final accuracy (S3)	66.21%	79.02%	+12.81pp
Accuracy span across S1-S3	5.42 pp	0.91 pp	FedTGP more consistent
Mean worst-client accuracy	49.32-57.38%	73.63-74.85%	FedTGP better at the low end
Runtime overhead	—	+33 to +41%	FedTGP more expensive
Seed standard deviation	1.58-3.30pp	0.44-0.83 pp	FedTGP less seed-sensitive

Table D.6: Summary of stress-test findings.

The experimental controls strengthen this interpretation. Both methods used the same saved data partition, corresponding runs began with matching client-model weights, and the training budget remained fixed. These decisions decreased the risks of any variation being due to a more favourable partition or a different initialization. Reporting accuracy after 100 completed rounds avoided selecting the highest-scoring test checkpoint for this comparison.

However, the architecture changes did not isolate model heterogeneity as a single independent factor. Replacing a network also changed its capacity, optimisation behaviour and computational requirements. Furthermore, the replacements were made on specific clients: client 4 changed in S2, and client 3 changed in S3. Both clients had substantial test sets. Repeating the experiment with the replacement architectures assigned to different clients would help determine how much the findings depend on these particular assignments.

The study also used one fixed partition and three training seeds. The reported standard deviations describe training variation under that partition, rather than uncertainty across possible federated datasets. Further partition seeds would be required to test if the same pattern occurs if the partition allocations are altered and or the number of samples collected for the various clients is altered. The results are only to be presented as descriptive experimental evidence, and not as a claim of general or statistically proven superiority. These results are descriptive experimental evidence, not statistically proven or general superiority. There were only three training trials on one fixed partition.

A further limitation concerns reproducibility. Random seeds were controlled, but the execution produced a warning that a CUDA operation did not have a deterministic implementation. The runs therefore followed a reproducible configuration, although exactly matching every numerical result across executions cannot be guaranteed.

Conclusion

Overall, the published evidence and practical experiments support the usefulness of FedTGP's learned global prototypes under the evaluated conditions. In the stress test, FedTGP achieved higher final accuracy and better mean worst-client performance than FedProto across all three architecture configurations, while requiring more computation. These findings support the hypothesis under the tested conditions, but do not establish superiority across other partitions or applications. Multiple prototypes per class and robust prototype aggregation remain untested alternatives that would require controlled evaluation.

Implementation and Evidence

The implementation is based on the author's FedTGP repository [6]. PFLlib was involved in the initial data-pipeline analysis [7] and HtFLlib was used to implement the heterogeneous FedProto in the practical comparison [8]. The baseline is based on the prototype-sharing method proposed by Tan et al. [9]. Tables D.3–D.6 and Figures D.1–D.3 are based on the results in the completed stress-test notebooks.

Coursework Repository: *https://github.com/ThisumiWijesinghe/FedTGP-Paper-Review*

Stress-test notebooks and results:

https://drive.google.com/drive/folders/1ARtsiMccEheQW2QF71Ki2S8xpUyOaNFj?usp=sharing

References

[1] J. Zhang, Y. Liu, Y. Hua, and J. Cao, "FedTGP: Trainable Global Prototypes with Adaptive-Margin-Enhanced Contrastive Learning for Data and Model Heterogeneity in Federated Learning," *Proc. AAAI Conf. Artif. Intell.*, vol. 38, no. 15, pp. 16768–16776, Mar. 2024, doi: 10.1609/aaai.v38i15.29617.

[2] W. A. T. Nimsani, "Critical review and technical baseline of a machine learning research paper: Phase 1 resubmission," unpublished coursework report, Machine Learning related Application, Oct. 12, 2026.

[3] W. A. T. Nimsani, "Critical review and technical baseline of a machine learning research paper: Phase 2 baseline implementation," unpublished coursework report, Machine Learning related Application, Oct. 15, 2026.

[4] K. Allen, E. Shelhamer, H. Shin, and J. Tenenbaum, "Infinite mixture prototypes for few-shot learning," in *Proc. 36th Int. Conf. Mach. Learn.*, PMLR, vol. 97, 2019, pp. 232–241. [Online]. Available: https://proceedings.mlr.press/v97/allen19b.html

[5] K. Pillutla, S. M. Kakade, and Z. Harchaoui, "Robust aggregation for federated learning," *IEEE Trans. Signal Process.*, vol. 70, pp. 1142–1154, 2022, doi: 10.1109/TSP.2022.3153135.

[6] TsingZ0, *TsingZ0/FedTGP*. (Jul. 27, 2026). Python. Accessed: Sep. 15, 2026. [Online]. Available: https://github.com/TsingZ0/FedTGP

[7] TsingZ0, *TsingZ0/PFLlib*. (Sep. 12, 2026). Python. Accessed: Sep. 14, 2026. [Online]. Available: https://github.com/TsingZ0/PFLlib

[8] TsingZ0, *TsingZ0/HtFLlib*. (Sep. 08, 2026). Python. Accessed: Sep. 15, 2026. [Online]. Available: https://github.com/TsingZ0/HtFLlib

[9] Y. Tan *et al.*, "FedProto: Federated Prototype Learning across Heterogeneous Clients," *Proc. AAAI Conf. Artif. Intell.*, vol. 36, no. 8, pp. 8432–8440, Jun. 2022, doi: 10.1609/aaai.v36i8.20819.